EMIPredict AI – Technical Documentation

Intelligent Financial Risk Assessment Platform – methodology, architecture and results

Live app: https://abhisheknimaje435-emipredict-ai-app-ccio5a.streamlit.app

Code repository: https://github.com/AbhishekNimaje435/emipredict-ai

1. Project overview

EMIPredict AI is a FinTech risk-assessment platform that solves two machine-learning problems on the same applicant profile:

- Classification: predict EMI eligibility as one of three classes – Eligible (low risk), High_Risk (marginal case) or Not_Eligible.
- Regression: predict max_monthly_emi, the maximum safe monthly EMI amount in INR.

All experiments are tracked and compared in MLflow, the best models are registered in the MLflow model registry, and the models are served through a multi-page Streamlit application deployed on Streamlit Cloud from a GitHub repository.

2. Dataset

Item	Value
Raw file	404,800 rows × 27 columns (25 input features + 2 targets)
After cleaning	400,024 rows
EMI scenarios	E-commerce Shopping, Home Appliances, Vehicle, Personal Loan, Education
Class distribution	Not_Eligible 77.3%, Eligible 18.4%, High_Risk 4.3%
Regression target	max_monthly_emi in INR, heavily right-skewed, from near zero up to about ₹90,000
Split (stratified, 70/15/15)	Train 280,016 / Validation 60,004 / Test 60,004

The 25 input features are grouped as personal demographics (age, gender, marital_status, education), employment and income (monthly_salary, employment_type, years_of_employment, company_type), housing and family (house_type, monthly_rent, family_size, dependents), monthly obligations (school_fees, college_fees, travel_expenses, groceries_utilities, other_monthly_expenses), financial status (existing_loans, current_emi_amount, credit_score, bank_balance, emergency_fund) and the loan application (emi_scenario, requested_amount, requested_tenure).

The strong class imbalance (High_Risk is only about 4% of the data) shaped the training and model-selection strategy described below.

3. Data quality assessment and preprocessing

The quality report on the raw data found the following issues, each handled in step1_preprocessing.py:

Issue found	Treatment
Missing values in education (2,404), monthly_rent (2,426), credit_score (2,420), bank_balance (2,426), emergency_fund (2,351)	Median imputation for numeric columns, mode imputation for categorical columns
age, monthly_salary and bank_balance stored as text (object dtype) because of noisy entries	Non-numeric characters stripped, then converted to numbers
Inconsistent gender labels: F, FEMALE, Female, female, M, MALE, Male, male	Normalised to two canonical values, Female and Male
Invalid credit_score values (above the valid 300–850 range, up to 1200)	Rows outside 300–850 removed (about 4,800 rows overall, 1.2% of the data)
Extreme values in salary, balances and requested amount	Clipped to the 1st–99th percentile

After cleaning, every categorical column contains only consistent labels. No duplicate rows were found.

4. Feature engineering

The feature logic lives in one function, add_features(), which is used both in training and in the live app so that predictions use exactly the same transformations.

- Financial ratios: total expenses, debt-to-income, expense-to-income, disposable income, affordability ratio, requested EMI estimate (amount / tenure), EMI-to-income, EMI-to-disposable-income, loan-to-annual-income.
- Savings and risk features: savings buffer in months, emergency-fund cover, credit risk score (credit score scaled to 0–1), employment stability (years employed relative to working age), dependents ratio.
- Interaction features: salary × credit risk score, debt-to-income × (1 − credit risk score), EMI burden × dependents.

Division by zero or missing values created by these ratios are replaced with zero, so the function never returns infinite or missing values.

Encoding and scaling: numeric features (the 17 original numeric inputs plus the 17 engineered ones, 34 in total) are standardised with StandardScaler; the 8 categorical features are one-hot encoded (handle_unknown set to ignore), giving 26 columns. Both are fitted on the training split only, to avoid data leakage. The result is 60 model input features.

5. Model development

Four models were trained for each problem: Logistic Regression, Decision Tree, Random Forest and XGBoost for classification, and Linear Regression, Decision Tree, Random Forest and XGBoost for regression. Hyper-parameters were chosen to keep the models accurate while keeping the stored model files small enough for deployment.

Class imbalance: all classification models were trained with balanced sample weights, so that the rare High_Risk class is not ignored. Because plain accuracy is misleading on imbalanced data, classification models are ranked by macro F1 (equal weight per class), and the recall of each class is logged as well.

5.1 Classification results (validation set)

Model	Accuracy	Macro F1	ROC-AUC	Recall High_Risk
Logistic Regression	0.8601	0.7140	0.9654	0.8411

Model	Accuracy	Macro F1	ROC-AUC	Recall High_Risk
Decision Tree	0.9403	0.8276	0.9681	0.8681
Random Forest	0.9209	0.7932	0.9859	0.8708
XGBoost (selected)	0.9752	0.9119	0.9969	0.9055

5.2 Regression results (validation set)

Model	RMSE (INR)	MAE (INR)	R^2	MAPE (%)
Linear Regression	3556.43	2428.13	0.7901	161.64
Decision Tree	1312.11	447.56	0.9714	10.10
Random Forest	1071.82	320.77	0.9809	8.42
XGBoost (selected)	704.24	239.51	0.9918	7.17

5.3 Model selection and justification

Classifier – XGBoost. It has the highest macro F1 and accuracy, and it recognises the rare High_Risk class best (about 90.6% recall on validation). Logistic Regression is a useful interpretable baseline but cannot capture the non-linear relationships in the financial ratios. Random Forest has a high ROC-AUC but a weaker macro F1.

Regressor – XGBoost. It has the lowest error on every metric. The large gap between Linear Regression (RMSE about 3,556) and the tree-based models shows that the maximum safe EMI depends on non-linear interactions between income, obligations and credit profile.

The selected models were also evaluated once on the held-out test set (60,004 records):

Task	Test-set result
Classification (XGBoost)	Accuracy 97.5%, macro F1 91.27%, ROC-AUC 99.7%, High_Risk recall 90.47%
Regression (XGBoost)	RMSE 746.17 INR, MAE 240.47 INR, R^2 0.9906, MAPE 7.26%

Targets from the project brief: classification accuracy above 90% – achieved (97.5% on the test set); regression RMSE below 2,000 INR – achieved (746 INR on the test set; 704 INR on the validation set).

6. MLflow experiment tracking and model registry

- Tracking store: SQLite backend (mlflow.db) with two experiments – EMI_Eligibility_Classification and EMI_Amount_Regression.
- Logged for every run: parameters and hyper-parameters, validation metrics, training time, a confusion matrix (classification), feature-importance plots for tree models, and the trained model artifact.
- Selection tags and test evaluation: the winning run of each experiment is tagged selected_for_production and is the only one evaluated on the test set.
- Model registry: the winners are registered as EMI_Eligibility_Classifier (run bdf73abb…) and EMI_Amount_Regressor (run 6076b8ae…), both version 1, and carry the production alias.
- Comparison: runs can be compared in the MLflow UI (mlflow ui --backend-store-uri sqlite:///mlflow.db) and in the Model Performance page of the app.

7. Application architecture

Application pages

1. Home – headline metrics of the selected models and navigation.
2. Prediction – real-time assessment of a single applicant (eligibility, confidence, class probabilities, maximum safe EMI, financial-health snapshot) and batch prediction from an uploaded CSV (up to 50,000 rows).
3. Data Explorer – interactive filters, distributions, relationships, engineered ratios and a correlation heatmap.
4. Model Performance – comparison of all eight models, selected-model highlights, confusion matrix, feature importance and registry details.
5. Admin Data Management – password-protected CRUD (create, read, update, delete) over financial records stored in a SQLite database; new records can have their targets auto-predicted by the deployed models.

Layers

Layer	Implementation
Data layer	Cleaned dataset (400,024 records); a 20,000-record sample ships with the app for exploration and the admin module
Processing layer	Cleaning pipeline, add_features(), fitted preprocessor (preprocessor.joblib), label encoder and feature list
Model layer	XGBoost classifier and regressor, exported in XGBoost's native JSON format; MLflow tracking and registry
Application layer	Multi-page Streamlit app
Deployment layer	GitHub repository connected to Streamlit Cloud (Python 3.12)

Prediction flow: raw applicant inputs are converted to numbers (missing numeric values are filled with the training median), passed through add_features(), transformed by the saved preprocessor, and scored by both XGBoost models. The classifier returns class probabilities and the label with the highest probability; the regressor output is clipped at zero.

8. Deployment

The code is hosted in a public GitHub repository and deployed on Streamlit Cloud, which rebuilds the app automatically when the repository changes. Library versions in requirements.txt are pinned to the versions used for training (scikit-learn 1.6.1, xgboost 3.4.1, numpy 2.1.3, pandas 2.2.3, joblib 1.6.0). The admin password is supplied through Streamlit secrets rather than stored in the repository.

9. Challenges and decisions

- Model serialisation: pickled XGBoost models failed to load on another machine (input stream corrupted). The boosters were exported to XGBoost's native JSON format, which is stable across versions; the exported models reproduced the original test

metrics exactly.

- Data consistency: the inconsistent gender labels were found while testing the app's dropdowns. The cleaning step was fixed, the models were retrained, and the metrics stayed essentially unchanged.
- Experiment hygiene: duplicate MLflow runs created by an accidental parallel run were removed so that each model has exactly one run.

10. Limitations and future work

- The admin database is a demo: on Streamlit Cloud its storage is temporary, and edits do not retrain the models. A production system would use a persistent database (for example PostgreSQL) plus a retraining pipeline.
- The requested EMI shown in the app ignores interest, so it is only an approximation of the real instalment.
- Further work: probability calibration, hyper-parameter search, fairness checks on demographic features such as gender, and model-drift monitoring.

This is an educational project; its predictions are not a substitute for real lending decisions.